

Policy Gradient (Finite Horizon)

Consider a (stationary) finite-horizon MDP with state space $\mathcal{S}$, **continuous action space** $\mathcal{A} \subseteq \mathbb{R}^{d_{\mathcal{A}}}$, transition kernel $\{p(\cdot|s, a) : s \in \mathcal{S}, a \in \mathcal{A}\}$, reward function $r : \mathcal{S} \times \mathcal{A} \rightarrow [-1, 1]$, starting-state distribution p_0 , and horizon $T \in \mathbb{N}$. We will consider stationary (stochastic Markovian) policies $\pi : \mathcal{S} \rightarrow \Delta_{\mathcal{A}}$ for simplicity, although the optimal policy for a finite-horizon MDP is typically non-stationary. Also, assume $r_T(s) = 0$ for $s \in \mathcal{S}$ (no terminal rewards).

While value-function approximation allows dealing with large and continuous *state* spaces, continuous actions require some sort of **policy approximation**. The simplest form of policy approximation is *action discretization*: divide $\mathcal{A}$ into a finite number K of bins and define a new, finite action space $\mathcal{A}' = [K]$ by defining a (state-dependent) representative action for each bin. This approach clearly suffers from the curse of dimensionality. However, the important thing to notice here is that discretizing the action space restricts the space of available (stochastic Markovian stationary) policies.

In general, consider a policy space or *policy class* $\Pi \subseteq \Delta_{\mathcal{A}}^{\mathcal{S}}$, where $\Delta_{\mathcal{A}}^{\mathcal{S}}$ denotes the space of all mappings from states to action distributions. Let $J : \Pi \rightarrow \mathbb{R}$ denote the expected return or *performance function*:

$$J(\pi) = \mathbb{E} \left[\sum_{t=0}^{T-1} r(S_t, A_t) \right],$$

where $S_0 \sim p_0$, $A_t \sim \pi(\cdot|S_t)$, and $S_{t+1} \sim p(\cdot|S_t, A_t)$ for $t = 0, \dots, T-1$. Let $\pi^* \in \arg \max_{\pi \in \Pi} J(\pi)$ denote the *best-in-class policy*, which may differ from the optimal policy of the MDP when Π is a proper subset of the space of all policies. Reinforcement learning algorithms that aim to find the best-in-class policy for a given policy space are called **policy search** or policy optimization algorithms. Typical policy spaces are parametric:

$$\Pi_{\Theta} = \{ \pi_{\theta} \in \Delta_{\mathcal{A}}^{\mathcal{S}} | \theta \in \Theta \},$$

where $\Theta \subseteq \mathbb{R}^d$ is the parameter space, and each parameter vector $\theta \in \Theta$ corresponds to a policy. A common example is that of policies parametrized by neural networks, where parameters are network weights. The mapping from parameter vectors to policies is not necessarily injective nor surjective. Since we will focus on parametric policies, we redefine the performance functions as $J(\theta) = J(\pi_{\theta})$. This is not just an abbreviation, as we are interested in $J : \Theta \rightarrow \mathbb{R}$ specifically as a function of parameters. Policy optimization reduces to the problem of finding the *optimal parameters* $\theta^* \in \arg \max_{\theta \in \Theta} J(\theta)$. Unfortunately, J is a nonconvex function in general. Nonconvexity can arise both from the structure of the policy optimization problem itself and from the parametrization. For simplicity, we consider $\Theta = \mathbb{R}^d$ in the following (unconstrained optimization).

Example 1 (Gaussian policies) Let $\mathcal{A} = \mathbb{R}$ (scalar continuous actions) and let $\mu_{\theta} : \mathcal{S} \rightarrow \mathbb{R}$ be a family of functions parametrized by $\theta \in \mathbb{R}^d$. We can define a class of stochastic policies for

this action space by specifying the probability density function (pdf) for each state and parameter vectors:

$$\pi_{\theta}(a|s) = \mathcal{N}(a; \mu_{\theta}(s), \sigma^2) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{1}{2\sigma^2}(a - \mu_{\theta}(s))^2\right),$$

where $\mathcal{N}$ denotes the pdf of the Gaussian distribution and $\sigma > 0$ is a (fixed) standard deviation. We call μ_{θ} the mean-action function. We shall see some concrete examples in the practical sessions. Gaussian policies can be easily generalized to multi-dimensional actions and learnable covariance matrices.

Example 2 (Softmax Policies) Although parametric policies find their natural application in continuous action spaces, we can easily define a parametric policy class over a finite action $\mathcal{A}$ by specifying the probability mass function for each state and parameter vector:

$$\pi_{\theta}(a|s) = \text{Softmax}(q_{\theta}(s, a)) = \frac{\exp(q_{\theta}(s, a))}{\sum_{a' \in \mathcal{A}} \exp(q_{\theta}(s, a'))},$$

where $q_{\theta} : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$, called the logit function, assigns a preference to each action for the given state.

Policy Gradient (PG) algorithms aim to solve the policy optimization problem by (stochastic) gradient ascent over policy parameters. Consider a class Π_{Θ} of stochastic (fully mixed) differentiable (with respect to parameters) policies. In particular, assume the following exists for all s, a, θ :

$$\nabla_{\theta} \log \pi_{\theta}(a|s) \in \mathbb{R}^d.$$

The latter is called **score function** or eligibility vector. For example, the score function for the Gaussian policy, assuming the action-mean function is differentiable, is

$$\nabla_{\theta} \log \pi_{\theta}(a|s) = \frac{a - \mu_{\theta}(s)}{\sigma^2} \nabla_{\theta} \mu_{\theta}(s),$$

and the score function of Softmax policies with differentiable logits is:

$$\nabla_{\theta} \log \pi_{\theta}(a|s) = \nabla_{\theta} q_{\theta}(s, a) - \mathbb{E}_{A \sim \pi_{\theta}(\cdot|s)} [\nabla_{\theta} q_{\theta}(s, A)].$$

Under mild regularity assumptions, we can define the gradient of the performance function, $\nabla_{\theta} J$, often called *policy gradient*. A simple policy gradient scheme is the following: starting from some initial parameters θ_0 , iterate the following parameter update:

$$\theta_{k+1} = \theta_k + \alpha \nabla_{\theta} J(\theta_k),$$

for $k = 0, 1, \dots$, where $\alpha > 0$ is a *step size* or learning rate. Under mild regularity assumptions and a sufficiently small step size, this can be shown to converge, in the sense that $\|\nabla_{\theta} J(\theta_k)\| \rightarrow 0$ as $k \rightarrow \infty$. The result might be a local maximum (or even a minimum, or a saddle point). Anyway, computing the gradient of the performance function requires full knowledge of the MDP. In the following, we will see how to estimate the policy gradient from on-policy data in a model-free fashion.

REINFORCE. Denote by $\tau = (S_0, A_0, \dots, S_{T-1}, A_{T-1}, S_T)$ a *trajectory*, that is the sequence of states and actions observed during an episode. Every policy π induces a distribution over trajectories, whose pdf can be written as follows:

$$p_\pi(\tau) = p_0(S_0)\pi(A_0|S_0)p(S_1|S_0, A_0) \cdots p(S_T|S_{T-1}, A_{T-1}) = p_0(S_0) \prod_{t=0}^{T-1} \pi(A_t|S_t)p(S_{t+1}|S_t, A_t).$$

For parametric policies, we abbreviate $p_{\pi_\theta}(\tau)$ as $p_\theta(\tau)$. The *return* of a trajectory is defined as:

$$R(\tau) = \sum_{t=0}^{T-1} r(S_t, A_t).$$

It is easy to see that the performance function (or expected return) can be written as follows:

$$J(\theta) = \mathbb{E}_{\tau \sim p_\theta} [R(\tau)].$$

Theorem 3 (Finite-Horizon Policy Gradient Theorem)

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim p_\theta} \left[\left(\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(A_t|S_t) \right) R(\tau) \right].$$

PROOF. We prove this and other results from this chapter for the case in which $\mathcal{S}$ and $\mathcal{A}$ are both finite. Let $\mathcal{T}$ denote the set of all possible trajectories of length T , which is also finite. The results generalize to continuous spaces under some regularity assumptions. Here and in the following, we omit the gradient subscript when clear from context.

$$\begin{aligned} \nabla J(\theta) &= \nabla \mathbb{E}_{\tau \sim p_\theta} [R(\tau)] \\ &= \nabla \sum_{\tau \in \mathcal{T}} p_\theta(\tau) R(\tau) \\ &= \sum_{\tau \in \mathcal{T}} \nabla p_\theta(\tau) R(\tau) \\ &= \sum_{\tau \in \mathcal{T}} p_\theta(\tau) \nabla \log p_\theta(\tau) R(\tau) \\ &= \mathbb{E}_{\tau \sim p_\theta} [\nabla \log p_\theta(\tau) R(\tau)], \end{aligned}$$

where in the fourth equality we used the *log trick* or likelihood-ratio trick: since $\nabla_x \log f(x) = \nabla f(x)/f(x)$, $\nabla_x f(x) = f(x) \nabla_x \log f(x)$ whenever $\nabla_x \log f(x)$ is defined. The purpose of this was to rewrite the policy gradient as an expectation under the same trajectory distribution as the performance function. Finally:

$$\begin{aligned} \nabla \log p_\theta(\tau) &= \nabla \log \left(p_0(S_0) \prod_{t=1}^{T-1} \pi_\theta(A_t|S_t) p(S_{t+1}|S_t, A_t) \right) \\ &= \nabla \left(\log p_0(S_0) + \sum_{t=0}^{T-1} \log \pi_\theta(A_t|S_t) + \sum_{t=0}^{T-1} \log p(S_{t+1}|S_t, A_t) \right) \\ &= \sum_{t=0}^{T-1} \nabla \log \pi_\theta(A_t|S_t), \end{aligned}$$

since all the terms that do not depend on policy parameters vanish. $\square$

This expression for the policy gradient allows us to immediately define an unbiased *Monte Carlo* gradient estimator, called the **REINFORCE estimator**. Given a trajectory $\tau = (S_0, A_0, \dots, S_T)$ sampled from p_{θ} :

$$g(\theta; \tau) = \left(\sum_{t=0}^{T-1} \nabla \log \pi_{\theta}(A_t | S_t) \right) R(\tau).$$

This estimator has three important properties: (i) it is an unbiased estimate of the policy gradient, that is, $\mathbb{E}_{\tau \sim p_{\theta}}[g(\theta; \tau)] = \nabla J(\theta)$ when $\tau \sim p_{\theta}$, as is obvious from the previous theorem; (ii) it can be computed entirely from the observed states, actions, and rewards, without prior knowledge of the MDP; (iii) the data are naturally obtained by the interaction of the agent with the environment, as playing π_{θ} produces trajectories distributed according to p_{θ} . It is a common practice to aggregate multiple trajectories from the same policy to reduce the *variance* of the estimate:

$$\hat{\nabla} J(\theta) = \frac{1}{n} \sum_{i=1}^n g(\theta; \tau_i),$$

where $\tau^1, \dots, \tau^n$ are n i.i.d. trajectories from p_{θ} and n is called *batch size*. These can be obtained from n independent episodes of interaction, which may be simulated in parallel. The averaged estimator is clearly also unbiased, but its variance is smaller by a factor $1/n$. The **REINFORCE algorithm** performs *stochastic* gradient ascent using the REINFORCE estimator. Variants of REINFORCE constitute a family of algorithms known as **actor-only policy gradient algorithms**, where the “actor” is the policy. The name refers to the fact that these algorithms only employ policy approximation, without any value-function approximation, while methods employing both value-function and policy approximation are called “actor-critic” (covered in the next lectures).

Algorithm 1 REINFORCE

Input: Initial policy parameters θ_0 , step size α , batch size N .

- 1: **for** $k = 1, 2, \dots$ **do**
- 2: Collect a batch of n independent trajectories $\tau^1, \dots, \tau^n$ with π_{θ_k}
- 3: Compute

$$\hat{\nabla} J(\theta_k) = \frac{1}{n} \sum_{i=1}^n \left(\sum_{t=0}^{T-1} \nabla \log \pi_{\theta}(A_t^i | S_t^i) \right) R(\tau^i)$$

- 4: Update policy parameters as $\theta_{k+1} \leftarrow \theta_k + \alpha \hat{\nabla} J(\theta_k)$
 - 5: **end for**
-

Common heuristics from supervised learning, such as the ADAM optimizer, can be employed in practice, especially when policies are parametrized by deep neural networks. The training loop can be stopped after a fixed number K of iterations, or when the norm of the stochastic gradient falls below some small threshold. REINFORCE can be shown to converge *on average* using diminishing step sizes.

Pros and cons of policy gradient algorithms. Compared to other families of RL algorithms, the main advantages of policy gradient algorithms are listed below. We refer to methods relying

primarily on value-function approximation (such as DQN) as **value-based** as opposed to **policy-based** methods such as actor-only *and* actor-critic policy gradient algorithms.

1. Naturally support continuous actions.
2. Well-understood (local) convergence guarantees even if function approximation is involved.
3. Robustness to noise: compared to value-based methods, actions (action distributions) change less abruptly when the states or the parameters are perturbed.¹ This typically yields more stable learning.
4. Naturally support stochastic policies, which are necessary in partially observable and strategic environments, and provide a minimal amount of exploration.
5. Suffer less from violations of the Markov assumption since it is not exploited directly (at least by actor-only algorithms).
6. Easy to incorporate domain knowledge: the policy space can be designed to only include behaviors relevant to the application (e.g., controllers with a small number of tunable parameters).
7. Safety: the policy space can be designed to exclude unsafe behaviors.

Some disadvantages are:

1. Variance: gradient estimators tend to have high variance, which makes the convergence slow and requires large amounts of simulation data. We will see some variance-reduction techniques in the following.
2. Bias: the policy space may not include good policies if it is not designed carefully.
3. Only convergence to local optima is guaranteed in general. Heuristics from nonconvex optimization such as random restarts can be employed. Global convergence can be ensured in some special cases (e.g., Linear Quadratic Regulator).
4. Do not naturally support deterministic policies. This can be a problem for safety-critical applications. Deterministic variants exist.

Variance Reduction (Basic Techniques). We now cover some basic techniques to reduce the variance of the REINFORCE estimator while keeping it unbiased. We will use the fact that scores are zero-mean conditional on the state:

$$\mathbb{E} [\nabla \log \pi_{\theta}(A|s)|S] = \sum_{a \in \mathcal{A}} \pi_{\theta}(a|S) \nabla \log \pi_{\theta}(a|S) = \sum_{a \in \mathcal{A}} \nabla \pi_{\theta}(a|S) = \nabla \sum_{a \in \mathcal{A}} \pi_{\theta}(a|S) = \nabla 1 = \mathbf{0},$$

¹Consider a (deterministic) greedy policy $\pi_{\theta}(s) = \arg \max_a Q_{\theta}(s, a)$, where Q_{θ} is some parametric approximation of Q^* . If we perturb the state or the parameters even slightly, the selected action might be completely different.

when $A \sim \pi_{\theta}(\cdot|S)$. This implies that $\mathbb{E}[\nabla \log p_{\theta}(\tau)] = \mathbf{0}$ when $\tau \sim p_{\theta}$, since:

$$\begin{aligned}\mathbb{E}[\nabla \log p_{\theta}(\tau)] &= \mathbb{E}\left[\sum_{t=0}^{T-1} \nabla \log \pi_{\theta}(A_t|S_t)\right] \\ &= \sum_{t=0}^{T-1} \mathbb{E}[\nabla \log \pi_{\theta}(A_t|S_t)] \\ &= \sum_{t=0}^{T-1} \mathbb{E}[\mathbb{E}[\nabla \log \pi_{\theta}(A_t|S_t)|S_t]] = \mathbf{0},\end{aligned}$$

where the third equality is by the *tower rule* or “law of conditional expectation”: $E[X] = E[E[X|Y]]$ for any random variables X, Y (not necessarily independent).

Besides using larger batch sizes, adding a **baseline** to the REINFORCE estimator is the simplest way to reduce the variance without introducing any bias. Let $b \in \mathbb{R}$ be any constant (independent of the batch of trajectories used to estimate the gradient). Consider the following **REINFORCE estimator with baseline**:

$$\hat{\nabla} J(\theta) = \frac{1}{n} \sum_{i=1}^n \left(\sum_{t=0}^{T-1} \nabla \log \pi_{\theta}(A_t^i|S_t^i) \right) (R(\tau^i) - b).$$

Theorem 4 *The REINFORCE estimator with baseline is unbiased.*

PROOF.

$$\begin{aligned}\mathbb{E}[\hat{\nabla} J(\theta)] &= \mathbb{E}\left[\frac{1}{n} \sum_{i=1}^n \nabla \log p_{\theta}(\tau^i) (R(\tau^i) - b)\right] \\ &= \mathbb{E}_{\tau \sim p_{\theta}}[\nabla \log p_{\theta}(\tau) (R(\tau) - b)] \\ &= \mathbb{E}_{\tau \sim p_{\theta}}[\nabla \log p_{\theta}(\tau) R(\tau)] - b \mathbb{E}_{\tau \sim p_{\theta}}[\nabla \log p_{\theta}(\tau)] \\ &= \nabla J(\theta) - \mathbf{0}.\end{aligned}$$

□

A common choice of baseline is the *average baseline*, which tends to reduce variance by centering the returns:

$$b = \frac{1}{n} \sum_{i=1}^n R(\tau^i).$$

In principle, this baseline should be computed from a separate, independent batch of trajectories. However, in practice, it is usually computed on the same batch of trajectories used for the gradient estimator. Common practice suggests that the bias is negligible if compared to the speedup granted by variance reduction and does not justify the cost of collecting extra trajectories.

A more subtle but still simple way to reduce variance is to exploit the temporal structure of the RL problem. Note that, in REINFORCE, $R(\tau)$ could be an arbitrary (possibly non-additive) function

of the trajectory, such as $R(\tau) = \max_{t=0, \dots, T-1} \{r(S_t, A_t)\}$. Indeed, REINFORCE can be used for non-Markovian processes and applications other than RL. On the downside, when applied to policy optimization in MDPs, it does not exploit the causal structure of trajectories at all. The **GPOMDP estimator**, on the contrary, exploits the Markov assumption to remove some terms that do not contribute to the expectation, but may be a source of variance:

$$\hat{\nabla} J(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n \sum_{t=0}^{T-1} \nabla \log \pi_{\boldsymbol{\theta}}(A_t^i | S_t^i) \sum_{h=t}^{T-1} r(S_h^i, A_h^i) = \frac{1}{n} \sum_{i=1}^n \sum_{t=0}^{T-1} r(S_t^i, A_t^i) \sum_{h=0}^t \nabla \log \pi_{\boldsymbol{\theta}}(A_h^i | S_h^i).$$

That the two expressions are equivalent can be checked by changing the order of summation. We will refer to them as the *first* and *second form* of GPOMDP.

Theorem 5 *The GPOMDP estimator is unbiased.*

PROOF. By the finite-horizon policy gradient theorem:

$$\begin{aligned} \nabla J(\boldsymbol{\theta}) &= \mathbb{E} \left[\left(\sum_{t=0}^{T-1} r(S_t, A_t) \right) \left(\sum_{t=0}^{T-1} \nabla \log \pi_{\boldsymbol{\theta}}(A_t | S_t) \right) \right] \\ &= \mathbb{E} \left[\sum_{t=0}^{T-1} \left(r(S_t, A_t) \sum_{h=0}^{T-1} \nabla \log \pi_{\boldsymbol{\theta}}(A_h | S_h) \right) \right] \\ &= \mathbb{E} \left[\sum_{t=0}^{T-1} \left(r(S_t, A_t) \sum_{h=0}^t \nabla \log \pi_{\boldsymbol{\theta}}(A_h | S_h) \right) \right] + \mathbb{E} \left[\sum_{t=0}^{T-1} \left(r(S_t, A_t) \sum_{h=t+1}^{T-1} \nabla \log \pi_{\boldsymbol{\theta}}(A_h | S_h) \right) \right]. \end{aligned}$$

If we can show that the second term is zero, then the first term shows that GPOMDP (better seen in the second form) is an unbiased Monte Carlo estimator of the policy gradient. Indeed:

$$\begin{aligned} &\mathbb{E} \left[\sum_{t=0}^{T-1} \left(r(S_t, A_t) \sum_{h=t+1}^{T-1} \nabla \log \pi_{\boldsymbol{\theta}}(A_h | S_h) \right) \right] \\ &= \mathbb{E} \left[\sum_{t=0}^{T-1} \sum_{h=t+1}^{T-1} \mathbb{E} [(r(S_t, A_t) \nabla \log \pi_{\boldsymbol{\theta}}(A_h | S_h)) | S_0, A_0, \dots, S_h] \right] \\ &= \mathbb{E} \left[\sum_{t=0}^{T-1} r(S_t, A_t) \sum_{h=t+1}^{T-1} \mathbb{E} [(\nabla \log \pi_{\boldsymbol{\theta}}(A_h | S_h)) | S_0, A_0, \dots, S_h] \right] \\ &= \mathbb{E} \left[\sum_{t=0}^{T-1} r(S_t, A_t) \sum_{h=t+1}^{T-1} \mathbb{E} [(\nabla \log \pi_{\boldsymbol{\theta}}(A_h | S_h)) | S_h] \right] \\ &= \mathbf{0}. \end{aligned}$$

We have used: in the first equality, linearity and the tower rule; in the second one, the fact that $r(S_t, A_t)$ is entirely determined by $S_0, \dots, S_h$, since $h > t$, and can be moved out of the conditional expectation; in the third one, the Markov property; in the last one, the fact that the score function is zero-mean conditional on the state. $\square$

Baselines can be added also to the GPOMDP estimator to further reduce variance, and can be made time-dependent:

$$\widehat{\nabla} J(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n \sum_{t=0}^{T-1} (r(S_t^i, A_t^i) - b_t) \sum_{h=0}^t \nabla \log \pi_{\boldsymbol{\theta}}(A_h^i | S_h^i).$$

In this case the average baseline is computed on rewards rather than returns:

$$b_t = \frac{1}{n} \sum_{i=1}^n r(S_t^i, A_t^i).$$

Natural Policy Gradient.* Parametric policy classes make policy updates *indirect*: we modify parameters to obtain an intended behavior. In most cases, the Euclidean metric in parameter space (standard distance in $\mathbb{R}^d$) does not reflect the entity of corresponding changes in policy space. For small displacements $\mathbf{h} \in \mathbb{R}^d$, the **Fisher metric** provides distances in parameter space that are more representative of the difference between the corresponding policies:

$$d_F(\boldsymbol{\theta}, \boldsymbol{\theta} + \mathbf{h}) = \|\mathbf{h}\|_{F(\boldsymbol{\theta})} = \sqrt{\mathbf{h}^\top F(\boldsymbol{\theta}) \mathbf{h}},$$

where $F(\boldsymbol{\theta}) \in \mathbb{R}^{d \times d}$ is the (trajectory-wise) **Fisher information matrix** of policy $\pi_{\boldsymbol{\theta}}$:

$$F(\boldsymbol{\theta}) = \mathbb{E}_{\tau \sim p_{\boldsymbol{\theta}}} \left[\nabla \log p_{\boldsymbol{\theta}}(\tau) \nabla \log p_{\boldsymbol{\theta}}(\tau)^\top \right],$$

a symmetric positive semi-definite matrix. Note that the Fisher metric is *local*: the distance between two points depends on where these points are located in parameter space. That is why, in most cases, it is only meaningful for small parameter displacements.

The (squared) Fisher metric provides an approximation of the *statistical distance* between the trajectory distributions induced by the corresponding policies (proof omitted):

$$D_{\text{KL}}(p_{\boldsymbol{\theta}+\mathbf{h}} \| p_{\boldsymbol{\theta}}) = \frac{1}{2} d_F(\boldsymbol{\theta}, \boldsymbol{\theta} + \mathbf{h})^2 + o(\|\mathbf{h}\|^3),$$

where D_{KL} is the **Kullback-Leibler divergence** or *relative entropy* of two distributions:

$$D_{\text{KL}}(p \| q) = \sum_{\tau \in \mathcal{T}} p(\tau) \log \frac{p(\tau)}{q(\tau)}.$$

This shows that the Fisher metric is (approximately) invariant to reparametrization: displacements under different but equivalent (that is, inducing the same behavior) parametrizations are measured in the same way. We can make this precise for affine parameter transformations:

Theorem 6 Consider a parametric policy class $\{\pi_{\boldsymbol{\theta}} \mid \boldsymbol{\theta} \in \mathbb{R}^d\}$ and a change of coordinates $\boldsymbol{\theta} = A\boldsymbol{\omega} + \mathbf{b}$, where $A \in \mathbb{R}^d$ is an invertible matrix and $\mathbf{b} \in \mathbb{R}^d$. Then, if $\boldsymbol{\theta}' = A\boldsymbol{\omega}' + \mathbf{b}$:

$$d_F(\boldsymbol{\omega}, \boldsymbol{\omega}') = d_F(\boldsymbol{\theta}, \boldsymbol{\theta}').$$

PROOF. First, notice that $p_{\boldsymbol{\theta}} = p_{\boldsymbol{\omega}}$ since $\boldsymbol{\theta}$ and $\boldsymbol{\omega}$ correspond to the same policy, hence induce the same trajectory distribution. By the chain rule:

$$\begin{aligned}\nabla_{\boldsymbol{\omega}} \log p_{\boldsymbol{\omega}}(\tau) &= \nabla_{\boldsymbol{\omega}} \log p_{\boldsymbol{\theta}}(\tau) \\ &= \frac{\partial \boldsymbol{\theta}}{\partial \boldsymbol{\omega}} \nabla_{\boldsymbol{\theta}} \log p_{\boldsymbol{\theta}}(\tau) \\ &= A^{\top} \nabla_{\boldsymbol{\theta}} \log p_{\boldsymbol{\theta}}(\tau).\end{aligned}$$

Then:

$$\begin{aligned}F(\boldsymbol{\omega}) &= \mathbb{E}_{\tau \sim p_{\boldsymbol{\omega}}} \left[\nabla_{\boldsymbol{\omega}} \log p_{\boldsymbol{\omega}}(\tau) \nabla_{\boldsymbol{\omega}} \log p_{\boldsymbol{\omega}}(\tau)^{\top} \right] \\ &= \mathbb{E}_{\tau \sim p_{\boldsymbol{\theta}}} \left[\nabla_{\boldsymbol{\omega}} \log p_{\boldsymbol{\omega}}(\tau) \nabla_{\boldsymbol{\omega}} \log p_{\boldsymbol{\omega}}(\tau)^{\top} \right] \\ &= \mathbb{E}_{\tau \sim p_{\boldsymbol{\theta}}} \left[A^{\top} \nabla_{\boldsymbol{\theta}} \log p_{\boldsymbol{\theta}}(\tau) \nabla_{\boldsymbol{\theta}} \log p_{\boldsymbol{\theta}}(\tau)^{\top} A \right] \\ &= A^{\top} F(\boldsymbol{\theta}) A.\end{aligned}$$

Finally, since $\boldsymbol{\omega}' - \boldsymbol{\omega} = A^{-1}(\boldsymbol{\theta}' - \boldsymbol{b}) - A^{-1}(\boldsymbol{\theta} - \boldsymbol{b}) = A^{-1}(\boldsymbol{\theta}' - \boldsymbol{\theta})$:

$$\begin{aligned}d_F(\boldsymbol{\omega}, \boldsymbol{\omega}') &= \sqrt{(\boldsymbol{\omega}' - \boldsymbol{\omega})^{\top} F(\boldsymbol{\omega}) (\boldsymbol{\omega}' - \boldsymbol{\omega})} \\ &= \sqrt{(\boldsymbol{\omega}' - \boldsymbol{\omega})^{\top} A^{\top} F(\boldsymbol{\theta}) A (\boldsymbol{\omega}' - \boldsymbol{\omega})} \\ &= \sqrt{(AA^{-1}(\boldsymbol{\theta}' - \boldsymbol{\theta}))^{\top} F(\boldsymbol{\theta}) AA^{-1}(\boldsymbol{\theta}' - \boldsymbol{\theta})} \\ &= \sqrt{(\boldsymbol{\theta}' - \boldsymbol{\theta})^{\top} F(\boldsymbol{\theta}) (\boldsymbol{\theta}' - \boldsymbol{\theta})} \\ &= d_F(\boldsymbol{\theta}, \boldsymbol{\theta}').\end{aligned}$$

□

Note that the two parametrizations define exactly the same policy space, so it is *natural* to measure distances in the same way under the two parametrizations. In contrast, the Euclidean metric would give:

$$\|\boldsymbol{\omega} - \boldsymbol{\omega}'\| = \|A^{-1}(\boldsymbol{\theta}' - \boldsymbol{\theta})\| = \sqrt{(\boldsymbol{\theta}' - \boldsymbol{\theta})^{\top} A^{-\top} A^{-1} (\boldsymbol{\theta}' - \boldsymbol{\theta})} = \|\boldsymbol{\theta}' - \boldsymbol{\theta}\|_{(AA^{\top})^{-1}},$$

which is *not* the same as $\|\boldsymbol{\theta}' - \boldsymbol{\theta}\|$ unless A is an orthogonal matrix ($AA^{\top} = A^{\top}A = I$), that is, unless the change of coordinates is an *isometry*.

Recall that the gradient gives the *steepest ascent* direction according to the Euclidean metric. The **natural gradient** gives the steepest ascent direction according to the Fisher metric (proof omitted), and is obtained by *preconditioning* the gradient with the inverse Fisher information matrix:

$$\tilde{\nabla} J(\boldsymbol{\theta}) = F(\boldsymbol{\theta})^{-1} \nabla J(\boldsymbol{\theta}).$$

An (exact) **natural policy gradient** update is then:

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k + \alpha \tilde{\nabla} J(\boldsymbol{\theta}_k).$$

It has two important properties, which are easily obtained from the properties of the Fisher metric we established above:

1. It allows to control the statistical distance between the updated policy and the original one. Namely, if $\mathbf{h} = \alpha \tilde{\nabla} J(\boldsymbol{\theta})$ and $\alpha = \epsilon / \|\tilde{\nabla} J(\boldsymbol{\theta})\|_{F(\boldsymbol{\theta})}$,

$$D_{\text{KL}}(p_{\boldsymbol{\theta}} \| p_{\boldsymbol{\theta}+\mathbf{h}}) = \frac{\epsilon^2}{2} + o(\|\mathbf{h}\|^3).$$

This is used to make sure the behavior of the agent does not change abruptly when updating the policy parameters with a small step size.

2. Invariance to parametrization. Consider a policy class $\{\pi_{\boldsymbol{\theta}} \mid \boldsymbol{\theta} \in \mathbb{R}^d\}$ and an invertible change of coordinates $\boldsymbol{\omega} = A\boldsymbol{\theta} + \mathbf{b}$. Let $\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k + \alpha \tilde{\nabla} J(\boldsymbol{\theta}_k)$ and $\boldsymbol{\omega}_{k+1} = \boldsymbol{\omega}_k + \alpha \tilde{\nabla} J(\boldsymbol{\omega}_k)$. If $p_{\boldsymbol{\theta}_k} = p_{\boldsymbol{\omega}_k}$, then $p_{\boldsymbol{\theta}_{k+1}} = p_{\boldsymbol{\omega}_{k+1}}$, hence also $J(\boldsymbol{\theta}_{k+1}) = J(\boldsymbol{\omega}_{k+1})$. The learning process is not affected by an affine reparametrization.

Practical implementations of natural policy gradient incur statistical and computational challenges: the Fisher information matrix must also be estimated from data, and inverting it may be computationally expensive for policies with many parameters.